Artificial Intelligence (AI)

Course Notes

Contains 10 PDF lessons

Generated: 3 May 2026

KnowledgeGate

Module

Artificial Intelligence

Topic

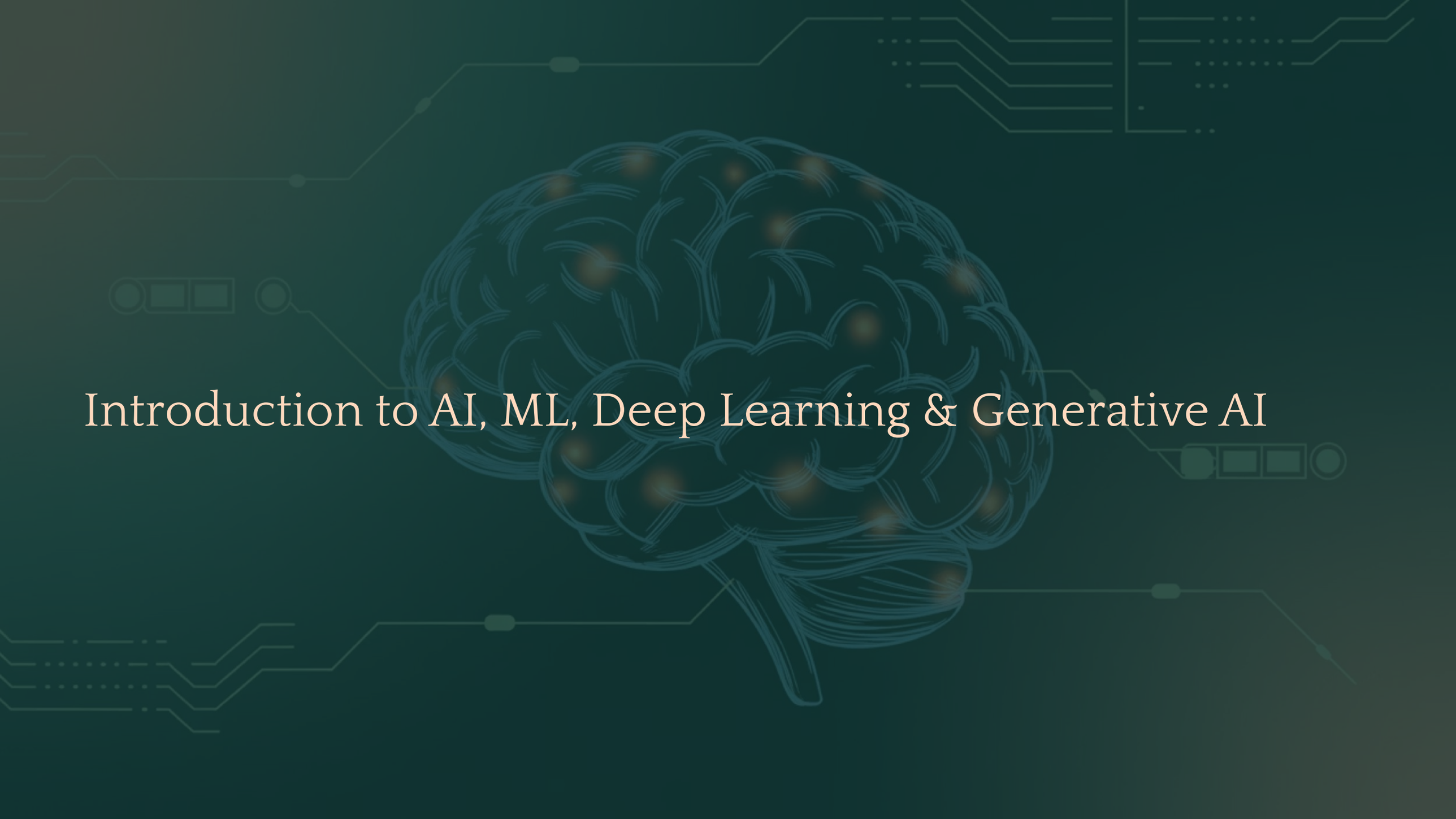
AI & ML by Arun Sir

Subtopic

GenAI Introduction

Lesson

[Notes] GenAI L1 - Short Intro


Introduction to AI, ML, Deep Learning & Generative AI

What You Will Learn Today

- What is Artificial Intelligence (AI)
- Machine Learning vs Deep Learning
- What is Generative AI?
- Traditional AI vs GenAI
- How LLMs work (simple explanation)
- Real-world applications

What is Artificial Intelligence?

AI = Machines performing tasks that require human intelligence.

Examples:



Google Maps



Face ID



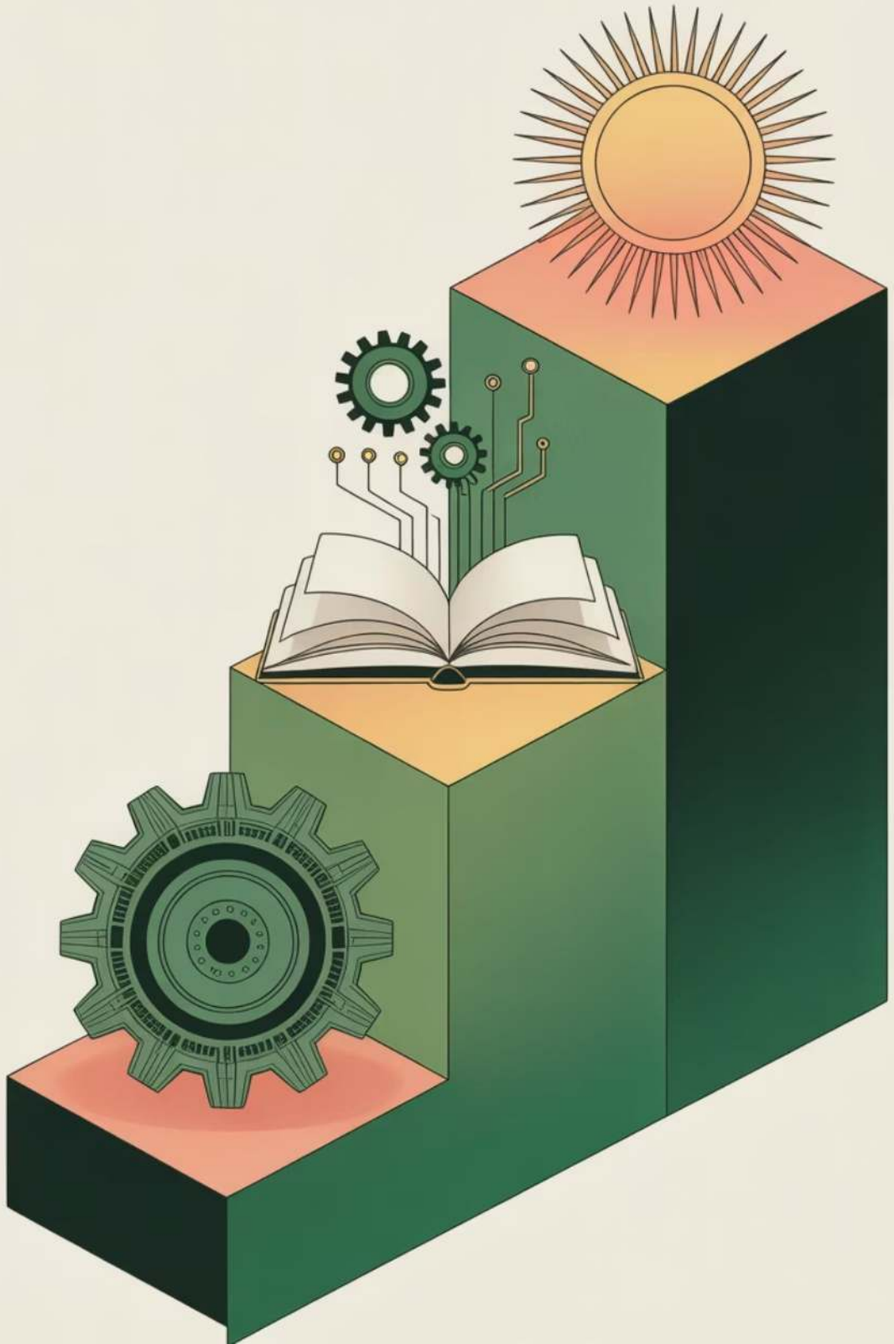
Spam detection



Voice assistants



Recommendation systems



Categories of AI

Narrow AI

designed for one task

General AI

human-like intelligence (not achieved yet)

Superintelligent AI

beyond human intelligence (theoretical)



What is Machine Learning (ML)?

ML = Machines learn patterns from data instead of being manually programmed.

Examples:

Spam vs Not Spam

Movie recommendations

Fraud detection

ML focuses on prediction.

How ML Works (Simple Flow)



Collect data

Clean and label data

Train model

Evaluate

Predict on new data

What is Deep Learning?

Deep Learning = Neural networks with multiple layers.

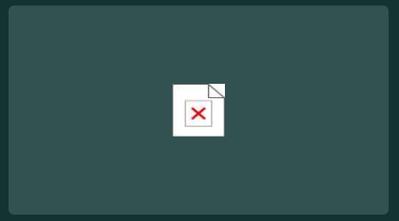
Used for:

- Speech recognition
- Image recognition
- Self-driving cars
- Chatbots

Deep Learning powered the GenAI revolution.



AI vs ML vs DL (Hierarchy)



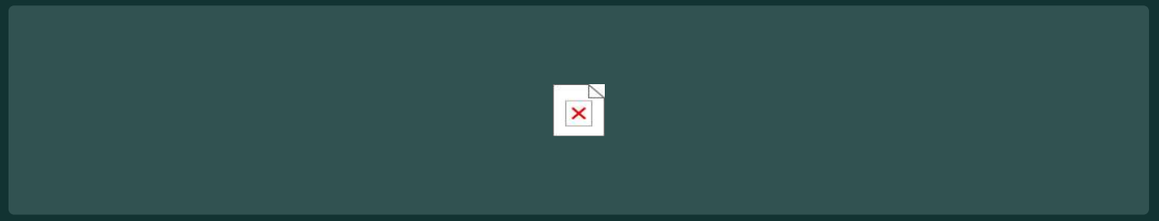
Deep Learning

Neural networks for high-dimensional tasks



ML

Learning from data



AI

Broad field

Limitations of Traditional AI

Traditional AI is:

- Predictive
- Classification based
- Limited creativity
- Dependent on labeled data
- Hard to scale for real-world text understanding



What is Generative AI?

GenAI = AI that creates new content.

Generates:

- Text
- Images
- Code
- Music
- Audio
- Video

Examples:

- ChatGPT
- Midjourney
- DALL·E
- Claude
- Copilot

Traditional AI vs Generative AI

Traditional AI:

- Predicts
- Classifies
- Detects
- Recommends

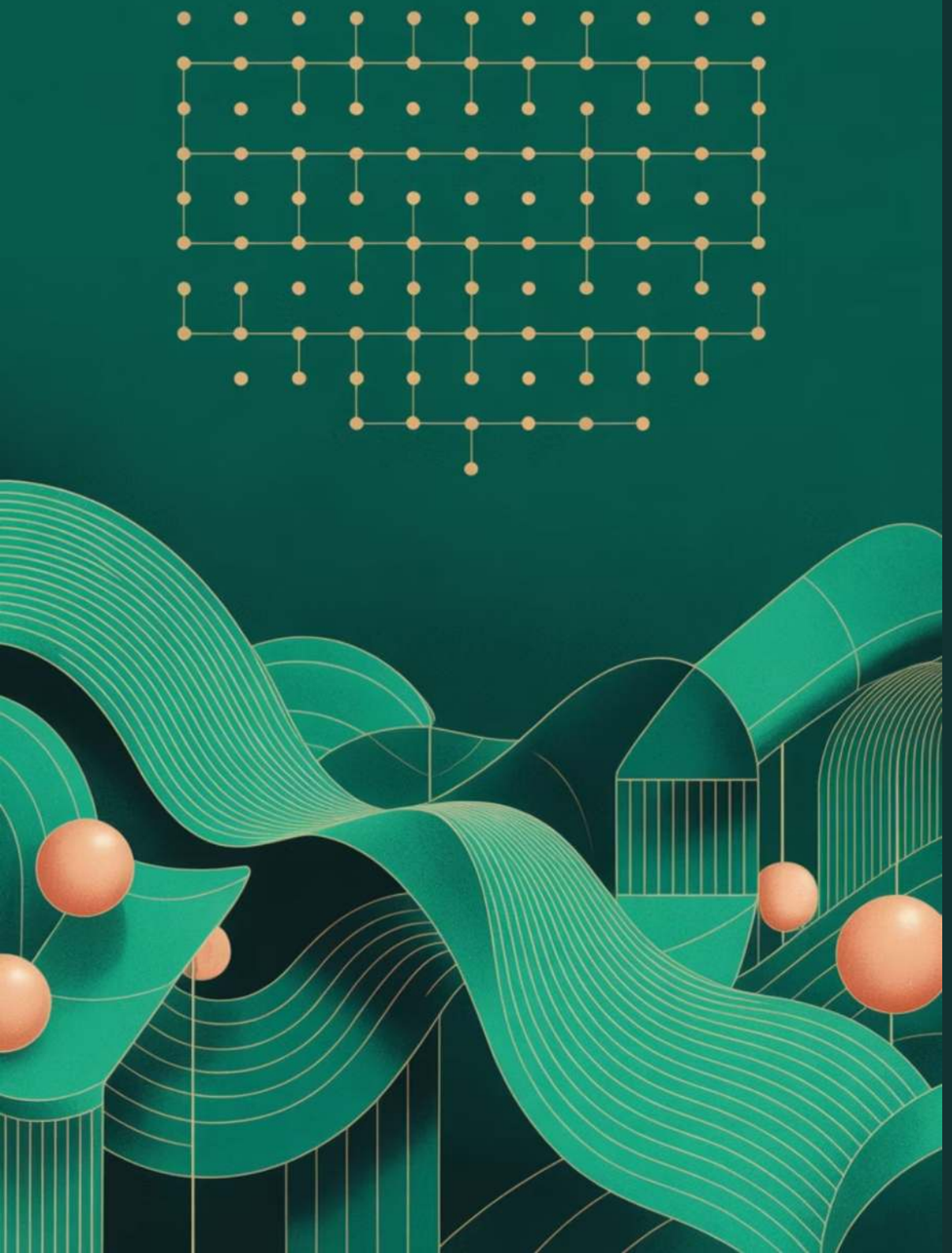
Generative AI:

- Creates
- Writes
- Designs
- Generates content

Why GenAI Became Big?

- Transformer architecture (2017)
- Self-attention mechanism
- Massive training data
- GPU advancements
- Cloud compute availability
- Foundation models

This combination created LLMs like GPT, Llama, Gemini, Claude.



How LLMs Work (Simple Explanation)

LLMs:

01

Read billions of words

02

Learn patterns, grammar, relationships

03

Generate text by predicting next best word

Not memorization — pattern generation.

Common LLM Examples

- GPT models (OpenAI)
- Llama models (Meta)
- Gemini (Google)
- Claude (Anthropic)
- Mistral / Mixtral

Applications of Generative AI

Business:

- Customer chatbots
- Document Q&A
- Email generation
- Meeting summaries

Developers:

- Code generation
- Debugging
- Documentation

Students:

- Notes summarisation
- Assignment explanation
- Resume building

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

GenAI Introduction

Lesson

[Notes] GenAI L2 - Creating first GenAI Program

Your First GenAI Program

Text Generation using Pre-trained LLMs

In this session, you will:

Run your **first Generative AI program**

Use a **pre-trained Large Language Model**

- Generate text from a simple prompt
- Understand what the model is doing (at high level)

Tools Used:

- Google Colab
- Python



What We Will Build Today

End Result of This Session

By the end of this lecture, you will be able to:

- **Generate text using a prompt**
- **Control output length and creativity**
- **Understand how LLMs generate responses**
- **Modify prompts to get different outputs**
- **Run GenAI code confidently in Colab**

📌 This is the foundation for chatbots, RAG systems, and AI apps.

What You Accomplished Today

- Ran your **first Generative AI program**
- Generated text using a **pre-trained LLM**
- Used **prompts** to control model output
- Understood how LLMs generate responses
- Built confidence working with GenAI tools

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

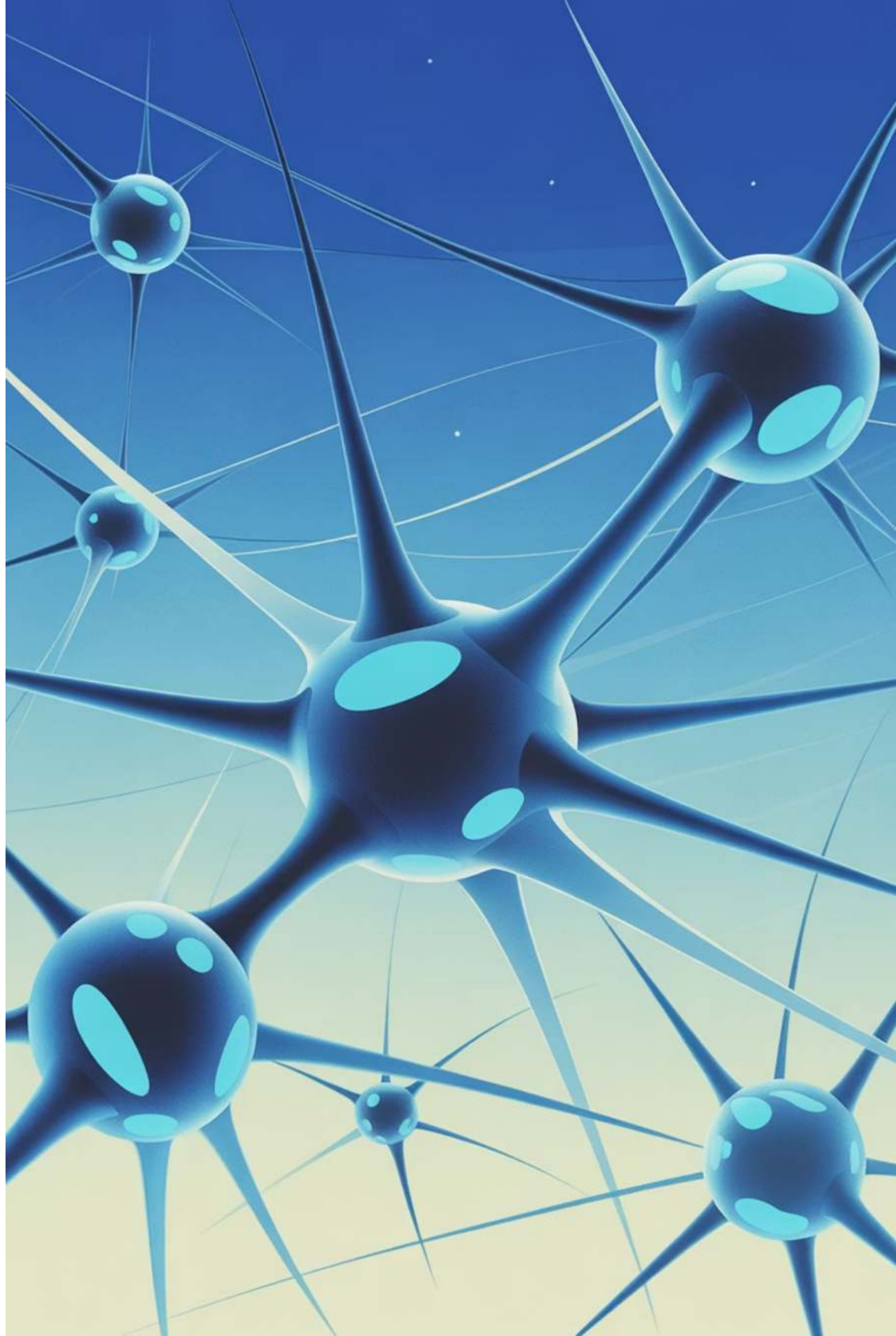
LLM's

Lesson

[Notes] GenAI L3 - LLMs

Large Language Models (LLMs)

How Machines Understand & Generate Language



What is an LLM?

Definition

A **Large Language Model (LLM)** is a sophisticated deep learning model trained on massive amounts of text data. These models are designed to understand context, patterns, and relationships within language, enabling them to generate human-like text responses.
Think of an LLM as a powerful pattern recognition system that has "read" billions of sentences and learned how language works—from grammar rules to nuanced context and meaning.



Why Are LLMs Called "Large"?

The term "large" in Large Language Models refers to their unprecedented scale across multiple dimensions. These models represent some of the most complex artificial intelligence systems ever created.



Billions of Parameters

Modern LLMs contain anywhere from 7 billion to over 175 billion parameters—the adjustable weights that the model uses to process information.



Massive Training Data

Trained on terabytes of text from books, websites, scientific papers, and more—representing a significant portion of human knowledge.



Intensive Computing Power

Require thousands of high-end GPUs running for weeks or months, consuming massive computational resources and energy.



Complex Language Patterns

Learn and encode millions of intricate language patterns, relationships, and contextual associations across diverse topics.

Core Idea: Next-Token Prediction

At their heart, LLMs work through a deceptively simple yet powerful mechanism: **predicting the next token in a sequence**. This fundamental process is what enables them to generate coherent, contextually relevant text.

1

Input Context

"Artificial Intelligence is"

2

Model Processing

Analyses patterns and context

3

Prediction

"changing"

4

Continuation

Repeats for next token

The model evaluates thousands of possible next tokens, assigns probabilities to each based on learned patterns, and selects the most likely candidate. This process repeats iteratively, building complete sentences, paragraphs, and even entire articles—one token at a time.

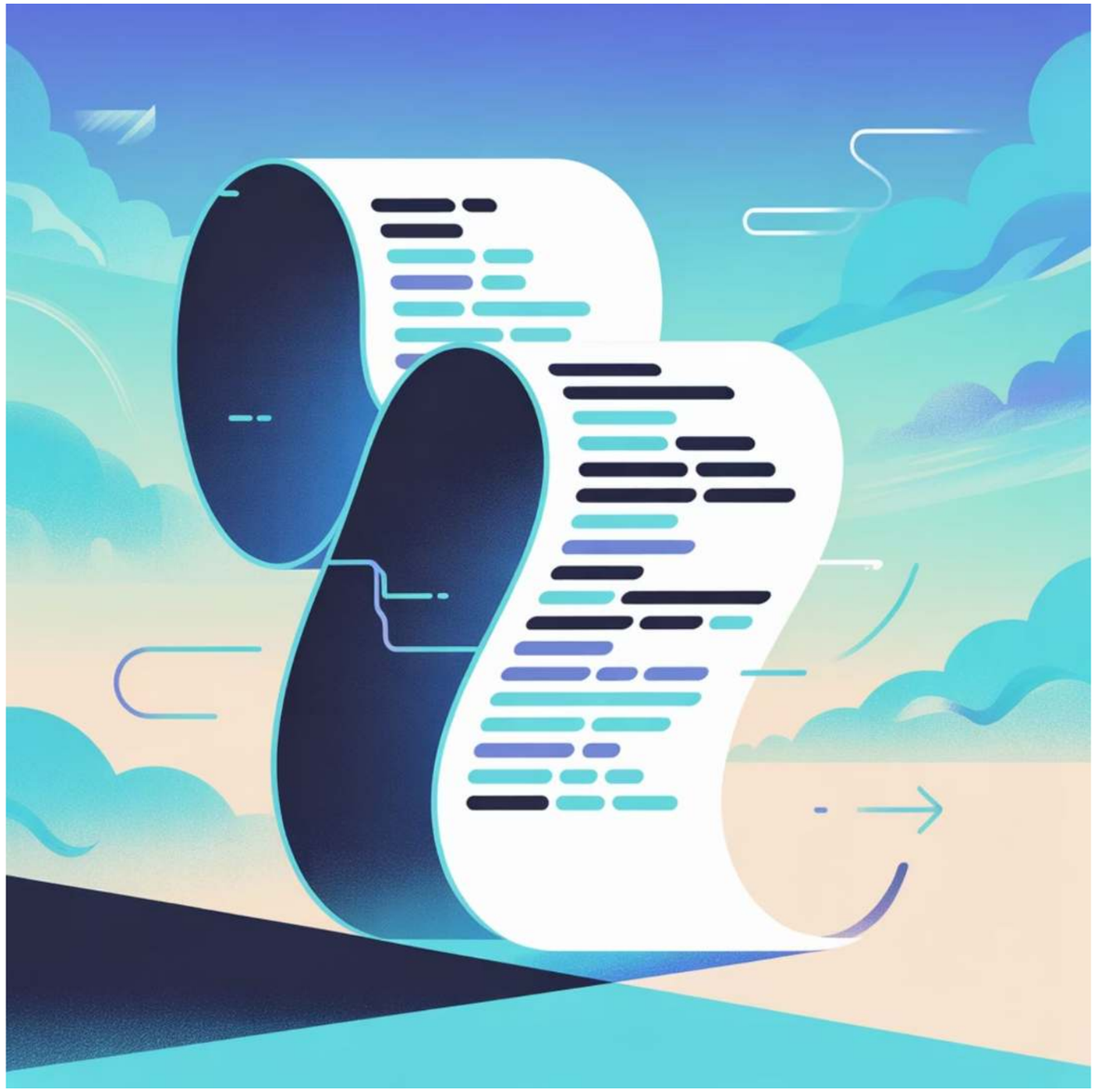


Tokens, Not Words

Important Concept

A crucial distinction: **LLMs do NOT process complete words** as humans do. Instead, they work with tokens—smaller units that can represent whole words, parts of words, or even individual characters.

For example, the word "understanding" might be split into tokens like ["under", "stand", "ing"]. This tokenisation allows the model to handle rare words, multiple languages, and even made-up terms efficiently.





How LLMs Are Trained

Training a Large Language Model is a multi-stage process that transforms a randomly initialised neural network into a sophisticated language understanding system. This journey involves two critical phases, each serving a distinct purpose.

1

Pre-training Phase

The foundation stage where the model learns general language patterns from vast amounts of raw text data.

2

Fine-tuning Phase

The specialisation stage where the model is refined for specific tasks and aligned with human preferences and safety requirements.

These two phases work together synergistically—pre-training provides broad knowledge and language understanding, whilst fine-tuning shapes the model's behaviour to be helpful, harmless, and honest.

Pre-training vs Fine-tuning

Pre-training



Huge Raw Text Data

The model ingests enormous datasets containing books, articles, websites, and more—often hundreds of billions of words representing diverse human knowledge.

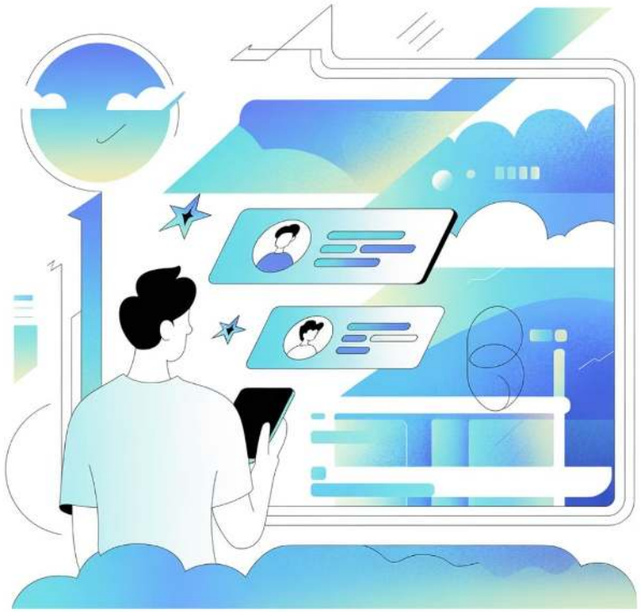
Self-supervised Learning

No human labels required. The model learns by predicting masked or next tokens, discovering language patterns independently through exposure to vast amounts of text.

Learns Grammar & Knowledge

Develops fundamental understanding of syntax, semantics, facts, and reasoning patterns embedded within the training corpus.

Fine-tuning



Smaller Curated Data

Uses carefully selected, high-quality datasets—often just thousands or millions of examples compared to billions in pre-training.

Human Feedback

Incorporates reinforcement learning from human feedback (RLHF), where human evaluators rate model responses to guide improvement in desired directions.

Instruction Following & Safety

Teaches the model to follow instructions accurately, avoid harmful content, admit limitations, and provide helpful, truthful responses.

Why LLMs Feel Intelligent

Large Language Models often create an impression of genuine intelligence and understanding. This perception emerges from several sophisticated architectural and training characteristics that enable remarkably human-like text generation.



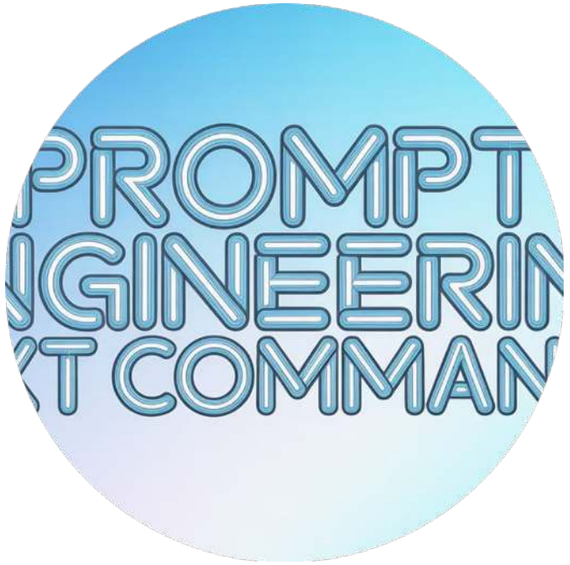
Self-Attention Mechanism

The Transformer architecture's self-attention allows the model to weigh the importance of different words in context, understanding relationships across long passages—similar to how humans track multiple concepts simultaneously.



Large-Scale Generalisation

Training on diverse, massive datasets enables the model to generalise patterns across domains, making connections between concepts it has never explicitly seen together—exhibiting emergent capabilities.



Task Versatility via Prompts

A single model can perform translation, summarisation, coding, analysis, and creative writing—all through natural language instructions, without requiring separate training for each task.

However, it's crucial to remember: LLMs are sophisticated pattern matchers, not conscious entities. They excel at statistical prediction but lack true understanding, reasoning, or awareness.

Capabilities & Limitations

Understanding both what LLMs can and cannot do is essential for leveraging them effectively whilst avoiding pitfalls. Let's examine their impressive strengths alongside their important constraints.

Capabilities

- **Text Generation**

  Create coherent, contextually relevant content across diverse styles—from formal reports to creative stories.

- **Summarisation**

  Condense lengthy documents into concise summaries whilst preserving key information and main ideas.

- **Translation**

  Translate between languages with impressive accuracy, capturing nuance and context better than earlier systems.

- **Code Generation**

  Write, debug, and explain code across multiple programming languages, accelerating software development.



Limitations

Hallucinations

Can generate plausible-sounding but entirely fabricated information, presenting false "facts" with confidence—especially problematic when knowledge gaps exist.

No Real-World Awareness

Lack genuine understanding of physical reality, current events (beyond training data cutoff), or personal experience—they cannot verify claims against the real world.

Training Data Dependency

Reflect biases, gaps, and inaccuracies present in training data. Cannot know information published after their training cutoff date.



Mental Model + Interview Tip

Mental Model for Understanding LLMs

Massive Pattern-Learning Engine

Statistical pattern recognition across billions of text examples

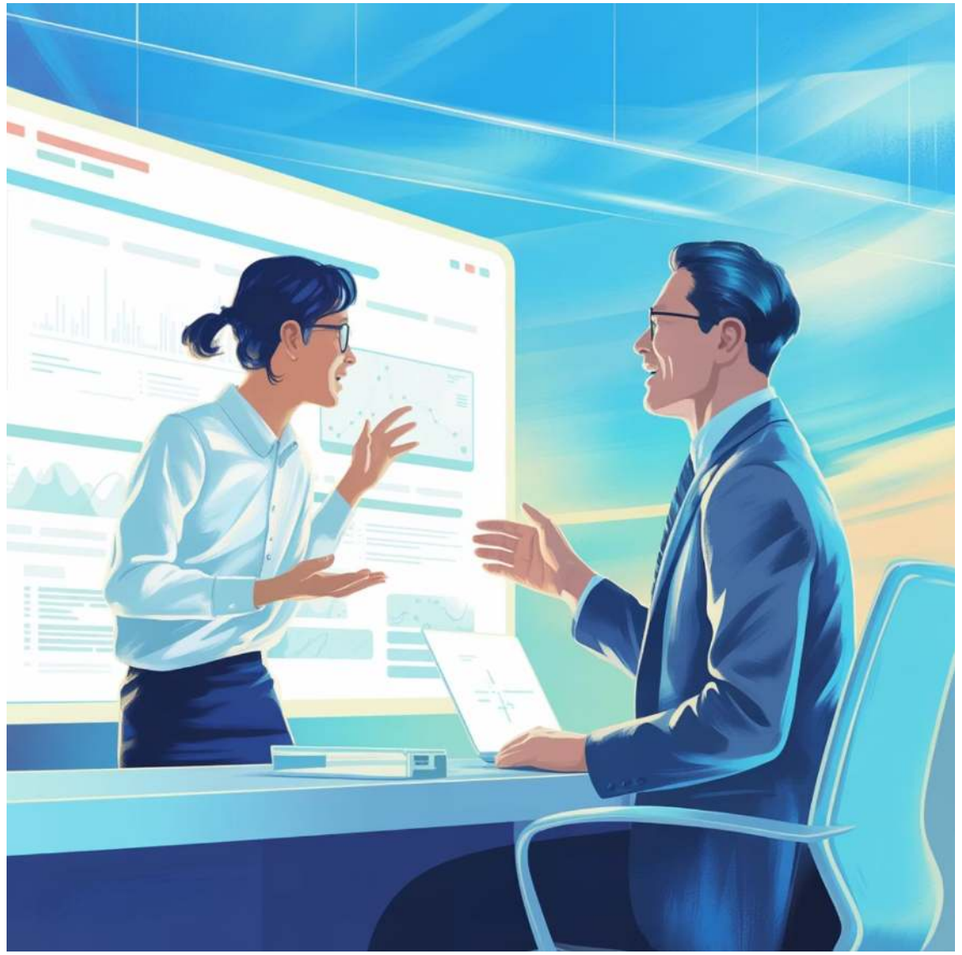
Combined With

Text Generation System

Iterative next-token prediction creating coherent output

This simple framework helps demystify LLMs: they're incredibly sophisticated pattern matchers that leverage scale to generate human-like text, not magical intelligence systems.

Interview Tip



Explain LLMs using this clear progression:

Tokens → How models process language

Next-Token Prediction → Core mechanism

Attention → Understanding context

Scale → Why size matters

This structured approach demonstrates deep understanding whilst remaining accessible to non-technical interviewers.

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

Tokenizations, Embeddings and Context Window

Lesson

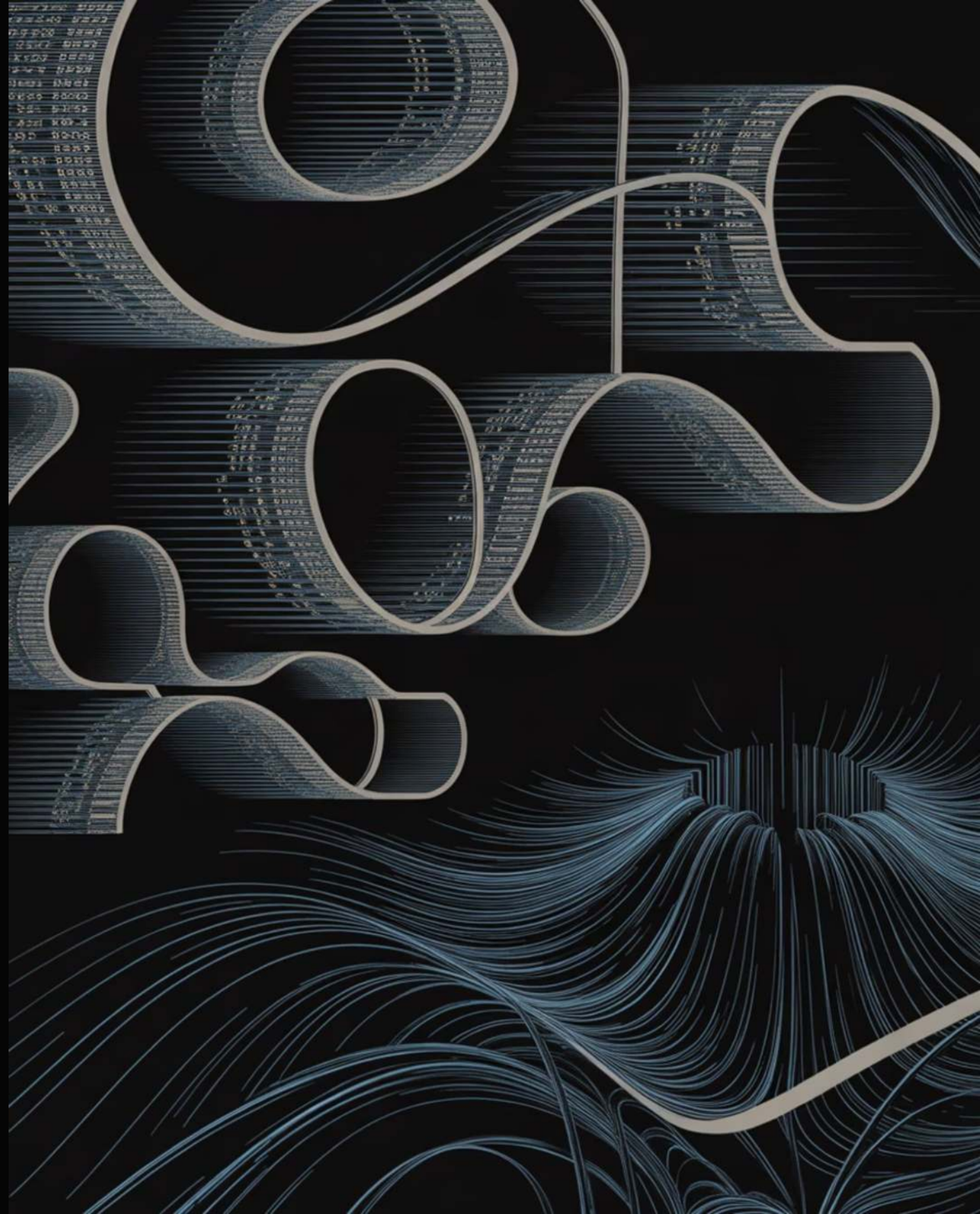
[Notes] GenAI L4 - Tokenizations, Embeddings and Context Window

Tokenization, Embeddings & Context Window

How LLMs Understand Your Text

Large Language Models (LLMs) don't understand text like humans. They convert text into numbers, learn patterns, and predict what comes next.

In this lecture, we'll understand **how raw text becomes meaningful output** inside an LLM — step by step.



Inside an LLM

Big Picture Flow

1. **Text**
   User input
2. **Tokens**
   Split into pieces
3. **Embeddings**
   Convert to numbers
4. **Model**
   Process patterns
5. **Output Tokens**
   Generate response
6. **Text**
   Final output

Example flow:

User types: *"Explain Kubernetes simply"*

LLM does NOT read English directly.

Instead:

- Text is split into tokens
- Tokens are converted to numbers
- Model processes numbers
- Output numbers are converted back to text

This flow is the **core pipeline of all LLMs**.

What is Tokenization?

Tokenization

Breaking text into smaller pieces called **tokens**

Tokens can be:

- Words
- Parts of words
- Characters or symbols

Example:

"ChatGPT is powerful" →

[Chat] [G] [PT] [is] [power] [ful]

Important point: Tokens ≠ words

One word can be multiple tokens.

Why Tokens Matter (Very Important)

Context length

How much text the model can remember

Cost

API pricing is charged per token

Model memory

Long conversations consume more tokens

Example:

- Short prompt → fewer tokens → cheaper
- Long document input → more tokens → higher cost

This is why prompt design matters in real-world applications.

Important Concept

From Tokens to Numbers

LLMs work only with numbers, not text.

Internal conversion pipeline:

01

Text → Tokens

02

Tokens → Token IDs

03

Token IDs → Embeddings

Example:

Token "apple" → Token ID: 12345 → Embedding: [0.12, -0.44, 0.98, ...]

Once converted to numbers, the model can apply math and probability.

Big Aha Slide

What Are Embeddings?

Embeddings

Numerical vectors that represent **meaning**

Key idea:

- Similar meanings → similar embeddings
- Different meanings → distant embeddings

Example:

- "king" and "queen" → close vectors
- "king" and "banana" → far apart

Embeddings allow models to **understand context, similarity, and intent**, not just keywords.



Why Embeddings Are So Powerful



Semantic search

Search by meaning, not exact words



RAG (Retrieval Augmented Generation)

Fetch relevant docs before answering



Recommendations

"users like you also viewed..."



Clustering

Group similar texts automatically

Example:

Search query: *"How to deploy app on cloud"*

Even docs with *"AWS EC2 deployment"* can be matched using embeddings.

What is Context Window?

Context Window

Maximum number of tokens a model can remember **at one time**

Example:

If context window = 8,000 tokens

Model can only see the **last 8,000 tokens** of conversation or input

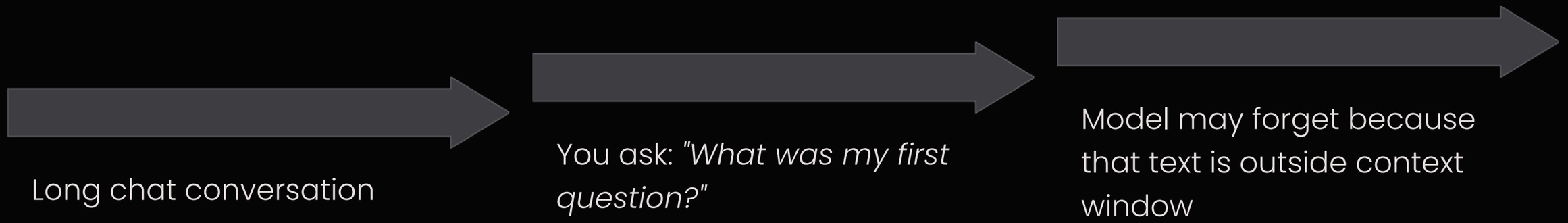
Older messages are ignored once the limit is crossed.

Think of it as **short-term memory** of the LLM.



Why Models 'Forget'

Reason: Limited context window

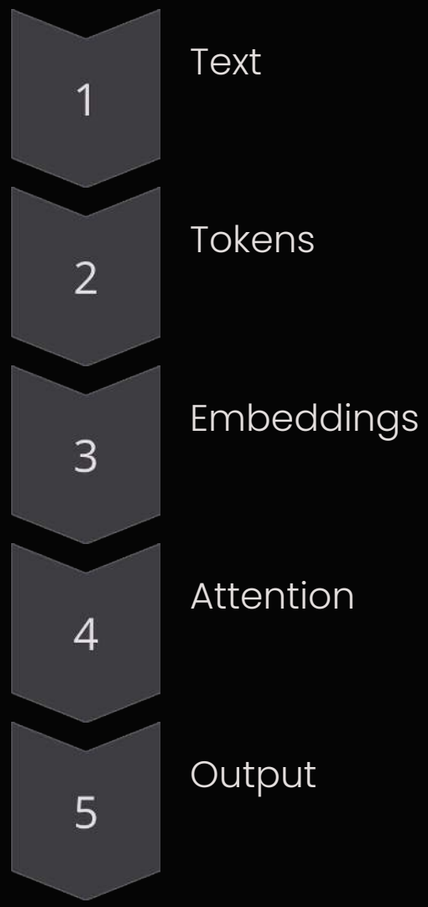


This is why:

- Chat history trimming
- Summarization
- External memory (RAG) are used in production systems.

Mental Model + Interview Tip

Mental Model



Always explain LLMs using this flow.

Interview Tip

If asked: *"How does GenAI work internally?"*

Explain:

1. Tokenization
2. Embeddings
3. Context window
4. Prediction of next tokens

This shows **conceptual clarity**, not just tool usage.

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

Transformers Architecture & Attention

Lesson

[Notes] GenAI L5 - Transformers Architecture & Attention

Transformers & Attention

The Engine Behind All Modern LLMs

How models like GPT, BERT, and LLaMA understand and generate language



The Problem Before Transformers

Older Models (RNN / LSTM)

Read text **one word at a time**

- Hard to remember information from far back in the sentence
- Long-range dependencies often lost

Training is **slow** (sequential processing)

Very difficult to **scale** on large datasets

Example of the Problem

> ### Sentence
>
> "The cat, which was very hungry, ate the food."

Challenge

The model must connect: 👉 **cat → ate**
RNNs may forget **"cat"** by the time they reach **"ate"**

This is called a long-range dependency problem



What Is a Transformer?

Key Idea

The model looks at the **entire sentence at once**

- Not strictly word-by-word
- Not strictly left-to-right

All tokens are processed **together**

This enables better understanding of context

What Is Attention?

For each word, the model decides: 👉

Important words get

Less relevant words get

Similar to how humans highlight important words while reading

Self-Attention (Core Concept)

Self-Attention

Each token looks at **all other tokens**

It assigns **importance scores**

- Context for each word is built using other words

This is the heart of the Transformer architecture

DEMO 1: Visualizing Attention (Conceptual)

Goal

Show that the model can connect words even when they are far apart

Concept

Attention layers show how strongly words relate to each other

Example:

- "it" attends strongly to "cat"
- "tired" attends to "it"

Self-attention allows long-distance connections

Why Self-Attention Is Powerful

- Long-range understanding of text
- Parallel processing of all tokens
- Much faster training compared to RNNs
- Better performance on large datasets

This is why Transformers replaced RNNs

Positional Encoding

Problem

Transformers do **not** naturally know word order

Solution

Add **positional information** to embeddings

- Position can be:
  - Sinusoidal (fixed)
  - Learned embeddings

This helps the model understand: 👉 "who came before whom"

Encoder vs Decoder

Original Transformer Architecture

LLMs like GPT

Use **Decoder-only** architecture

DEMO 2: Decoder-Only Text Generation

🎯 Goal

Connect transformer theory to real output

Concept

The model predicts the **next token**

Uses **masked self-attention**

- Can only see past tokens, not future ones

This is how text is generated step-by-step



Mental Model + Interview Answer

Mental Model

Tokens → Embeddings → Positional Info → Self-Attention → Output

Interview-Ready Answer

"Transformers use self-attention to understand relationships between all words in a sentence and generate text efficiently at scale."

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

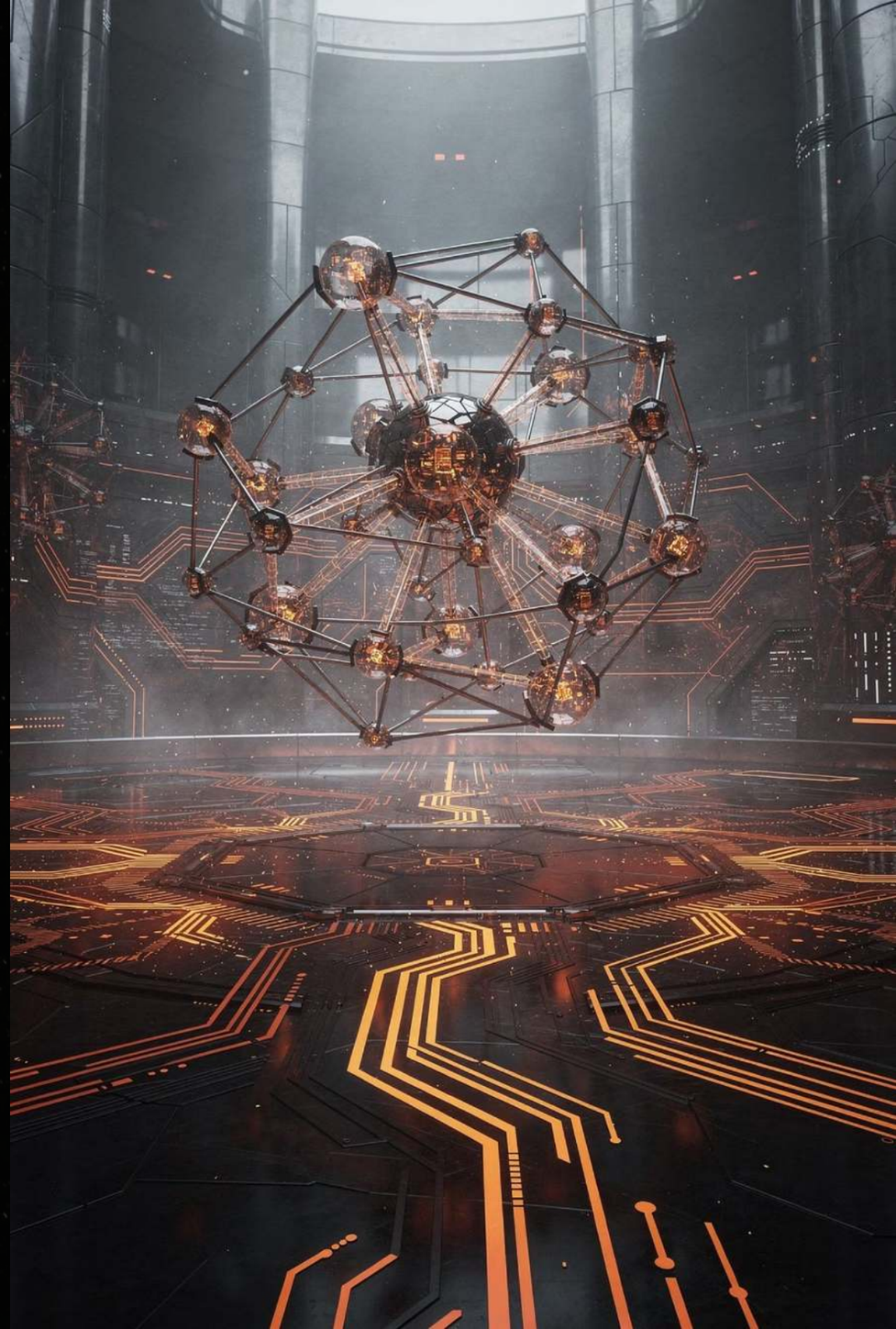
Prompt Engineering

Lesson

[Notes] GenAI L6 - Prompt Engineering

Prompt Engineering Fundamentals for GenAI Interviews

Master the art of communicating with Large Language Models



INTERVIEW ESSENTIALS

Why Prompt Engineering Matters

What is Prompt Engineering?

Prompt engineering is the practice of designing and refining input instructions to get the best possible output from Large Language Models (LLMs). It's about crafting clear, structured prompts that guide the AI to produce accurate, relevant, and useful responses.

Think of it as writing precise instructions for a highly capable but literal assistant—the better your instructions, the better the results.

Why It's Critical

- LLMs like GPT, Claude, and Gemini depend entirely on prompt quality
- Same model can give vastly different outputs based on how you ask
- Essential skill for GenAI engineers, ML practitioners, and software developers
- Frequently tested in technical interviews at AI companies

Interview Tip: Be ready to explain how prompt design impacts model performance and why it's crucial for production systems.

How LLMs Respond to Prompts

Understanding the flow from your input to the model's output helps you craft better prompts and troubleshoot issues effectively.

Your Prompt

The instruction or question you provide

Tokenization

Text is broken into tokens (words/subwords)

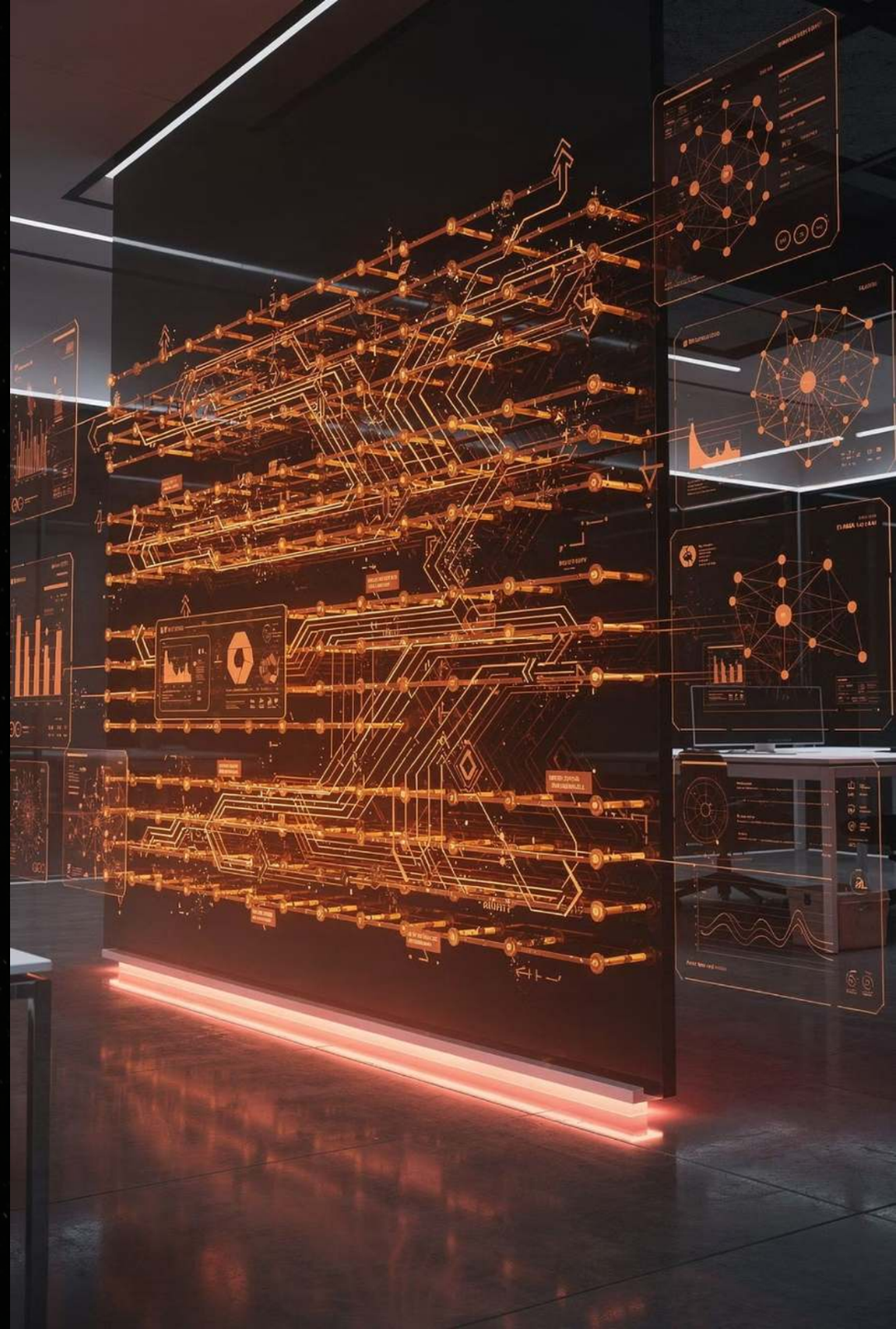
Model Processing

Neural network predicts next tokens based on patterns

Generated Output

Model produces response following your instructions

LLMs are instruction-following systems—they try to complete or respond to whatever pattern you establish. Clear instructions lead to predictable, high-quality outputs. Vague prompts result in inconsistent or irrelevant responses.



Anatomy of a Well-Structured Prompt

A good prompt has four key components. Understanding these building blocks helps you engineer effective prompts consistently.

1

Instruction

The task you want the model to perform. Be specific about the action: summarize, explain, generate, analyze, translate, etc.

2

Context

Background information that helps the model understand the situation, audience, or domain. This could include role, constraints, or domain knowledge.

3

Input Data

The actual content the model should process—text to summarize, code to review, question to answer, etc.

4

Output Format

How you want the response structured: JSON, bullet points, numbered steps, table, specific length, tone, etc.

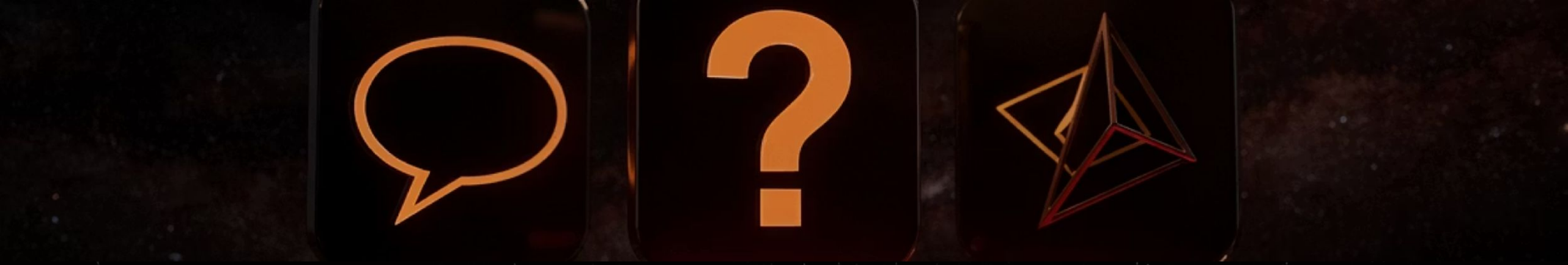
Example Prompt:

Instruction: Explain the concept of recursion

Context: to a final-year CS student preparing for interviews

Input: Focus on how it's used in tree traversal problems

Output Format: Provide a simple definition, one code example, and 2-3 interview tips. Keep it under 150 words.



Basic Prompt Types You'll Use

Question-Based

Direct questions to extract information

Example: "What are the differences between supervised and unsupervised learning?"

Best for: Quick facts, definitions, comparisons

Instruction-Based

Clear commands for specific tasks

Example: "Write a Python function to reverse a linked list."

Best for: Code generation, content creation, transformations

Role-Based

Assign the model a specific persona or expertise

Example: "Act as a senior DevOps engineer and review this deployment script."

Best for: Domain-specific advice, technical reviews, simulations

In interviews, you'll often be asked to demonstrate when each type is most appropriate. Understanding their strengths helps you choose the right approach for different scenarios.

Zero-Shot vs Few-Shot Prompting

These are fundamental techniques that dramatically affect model performance. Knowing when to use each is a common interview question.

Zero-Shot Prompting

Providing just the instruction without examples. The model relies entirely on its training.

```
Classify the sentiment: "This product exceeded my expectations!"
```

Output: Positive

When to use: Simple, well-known tasks; when examples aren't available; testing model capabilities

Few-Shot Prompting

Providing 2-5 examples before your actual task. Guides the model toward desired output pattern.

```
Classify sentiment:
"I loved it!" → Positive
"Terrible quality" → Negative
"Not bad" → Neutral

"This product exceeded my expectations!"
```

Output: Positive (with higher confidence)

When to use: Complex tasks; custom formats; domain-specific problems; when accuracy matters

Interview Insight: Few-shot prompting typically improves accuracy by 15-40% for classification tasks but increases token usage. Be ready to discuss this trade-off.

CRITICAL TECHNIQUE

Role Prompting: The Interview Game-Changer

Role prompting (also called system prompting) is one of the most powerful techniques. By assigning the model a specific role or persona, you significantly improve output quality and relevance.

Technical Expert

"Act as a senior backend engineer with 10 years of experience in distributed systems."

Educator

"Act as a computer science professor explaining concepts to undergraduate students."

Reviewer

"Act as a code reviewer focusing on security vulnerabilities and performance issues."

Interviewer

"Act as a technical interviewer at a FAANG company evaluating data structures knowledge."

Why Role Prompting Works

It activates specific knowledge domains in the model's training, adjusts tone and complexity appropriately, provides context for the type of response needed, and sets expectations for depth and style. In interviews, demonstrating understanding of role prompting shows maturity in working with LLMs.

Output Control Techniques

Controlling format, tone, and structure ensures the model's response matches your exact needs—crucial for production systems and APIs.

Format Control

"Provide the output as a JSON object with keys 'summary', 'sentiment', and 'confidence_score'."

"List 5 key points in numbered format."

"Generate a Python dictionary with error handling."

Tone Control

"Explain in simple language suitable for beginners."

"Use professional, formal tone for a technical document."

"Write in a conversational, friendly style."

Constraint Control

"Limit response to 100 words maximum."

"Provide exactly 3 examples, no more."

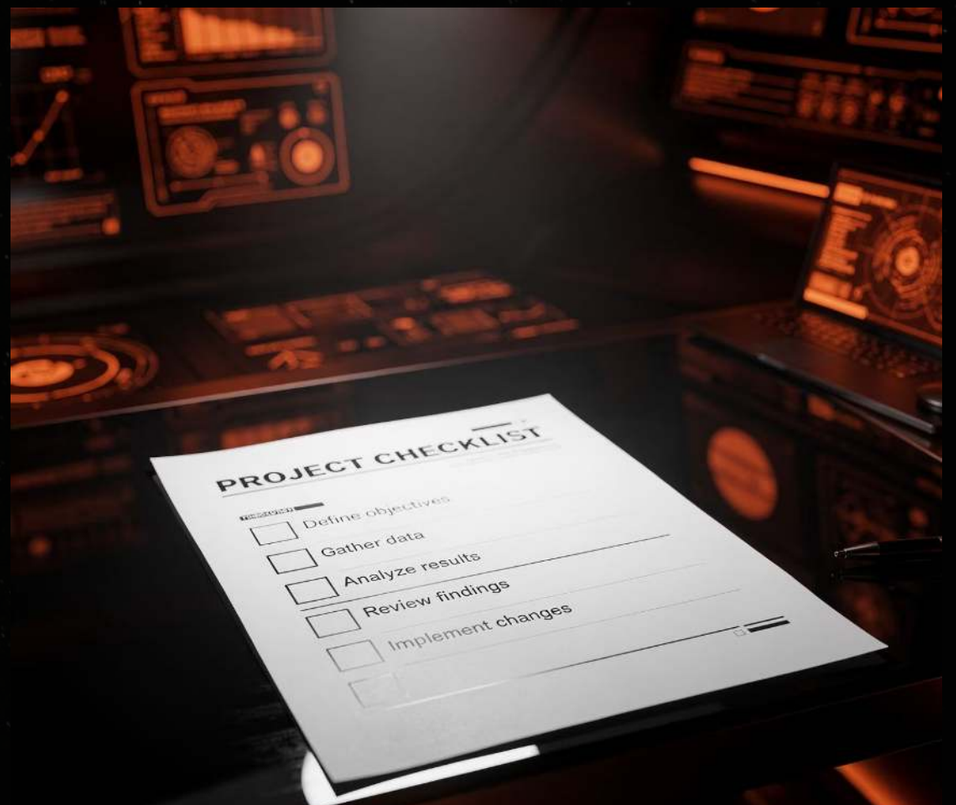
"Break solution into 5 clear steps."

Example: Controlled Output

Prompt: "Explain binary search in simple terms. Format as: 1) One-sentence definition, 2) Time complexity, 3) One use case. Keep total under 50 words."

Output:

1. Binary search finds elements in sorted arrays by repeatedly dividing search space in half
2. Time complexity: O(log n)
3. Use case: Finding a user ID in a sorted database of millions of records efficiently



Common Prompt Engineering Mistakes

Avoiding these pitfalls separates good prompt engineers from great ones. Interviewers often ask you to identify and fix problematic prompts.

01

Vague Instructions

Bad: "Tell me about databases"

Good: "Explain the differences between SQL and NoSQL databases, with one example of when to use each"

02

Missing Context

Bad: "Is this code good?"

Good: "Review this Python function for a REST API. Check for security issues, error handling, and performance. The function processes user authentication."

03

Overloading Instructions

Bad: "Explain ML, write code, compare algorithms, suggest resources, and create a study plan"

Good: Break into separate prompts: First explain ML, then request code, then compare algorithms

04

No Output Format

Bad: "List Python interview questions"

Good: "List 10 Python interview questions for senior developers. Format as: Question | Difficulty | Key Concept"

Quick Interview Prep: Key Questions & Takeaways

Top Prompt Engineering Interview Questions

- **What is prompt engineering and why is it important?** Focus on its role in controlling LLM behavior and ensuring reliable outputs
- **Explain zero-shot vs few-shot prompting with examples** Highlight trade-offs between simplicity and accuracy
- **How do you improve LLM output quality?** Discuss clear instructions, context, examples, format specification, and role prompting
- **What is role prompting and when is it useful?** Explain how personas activate domain knowledge and adjust tone
- **How do you handle inconsistent LLM responses?** Add constraints, use temperature control, provide examples, iterate on prompts

Key Takeaways

Clarity Over Length

Precise, structured prompts beat lengthy, vague ones every time

Structure Matters

Use instruction + context + input + format for consistent results

Essential GenAI Skill

Prompt engineering is fundamental for ML engineers, AI developers, and software engineers working with LLMs

Practice Tip: Try improving prompts on ChatGPT, Claude, or Gemini. Compare outputs before and after optimization. This hands-on experience is invaluable for interviews.

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

Hallucination in LLMs

Lesson

[Notes] GenAI L8 - Hallucination in LLMs



What is Hallucination in LLMs?

Understanding when AI generates confident but incorrect information

Definition

What is Hallucination?

Hallucination occurs when an LLM generates an answer that sounds confident and fluent but is factually incorrect, fabricated, or not grounded in reality.

- Sounds convincing
- Grammatically correct
- Confident tone
- But factually wrong or made-up

Simple Analogy (Human Example)

Analogy

A student in an exam:

- Does not know the answer
- Still writes something confidently
- Uses good language
- But the content is wrong

👉 This behaviour is **hallucination**.



Why Hallucination Happens (Core Reason)

> ### Important Reason
>
> LLMs do **not verify facts**. They predict the **next most likely word** based on patterns.

LLMs optimise for:

- What sounds like a good answer

❌ Not for:

- What is factually true

Simple Hallucination Example

1 2 3

Prompt

Who was the first Prime Minister of Mars?

Possible LLM Response

"The first Prime Minister of Mars was John Anderson, appointed in 2087."

What Happened?

- Answer sounds real
- Mars has no Prime Minister
- Information is completely fabricated

👉 This is hallucination.

Technical Hallucination Example

Prompt

Explain the Kubernetes feature XYZ-Controller.

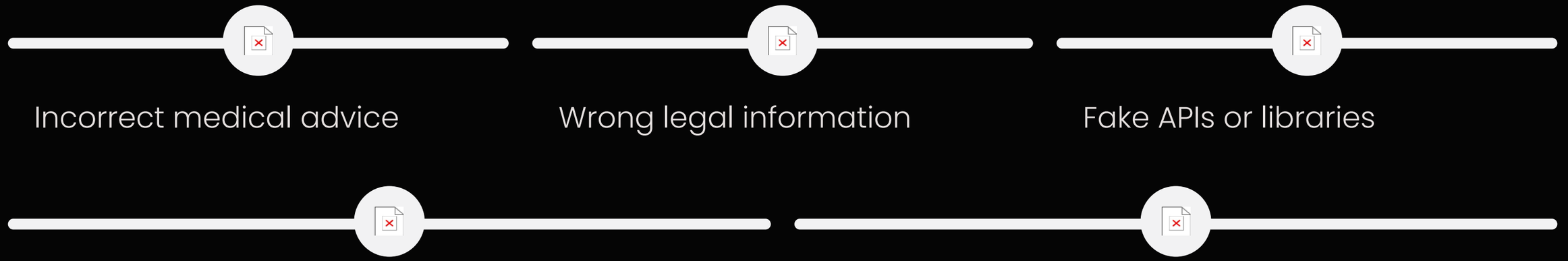
(Assume this feature does not exist)

Possible LLM Response

- Detailed explanation
- Architecture
- Benefits
- Use cases

❌ Feature is imaginary 👉 Technical hallucination

Why Hallucination Is Dangerous



Incorrect medical advice

Wrong legal information

Fake APIs or libraries

Wrong interview answers

Broken production systems

⚠️ Confidence does not mean correctness

Hallucination vs Error (Important Distinction)

Hallucination

- Fluent
- Confident
- Fabricated or unverified

Simple Error

- Obvious mistake
- Low confidence
- Easy to detect

👉 Hallucination is more dangerous because it **looks correct**.

When Hallucination Increases

Hallucination is more likely when:

- Prompt is vague
- Context is missing
- Question asks for unknown facts
- Model is forced to answer
- No constraints are given

How Engineers Reduce Hallucination (High Level)

Common Techniques

- Clear and specific prompts
- Output constraints (format, limits)
- Asking model to say "I don't know"
- Providing examples (few-shot)
- Using external knowledge (RAG)

Interview-Ready Definition

"Hallucination is when a large language model generates fluent but factually incorrect or fabricated information due to lack of grounding."

Key Takeaway

Hallucination is **not a bug**. It is a **natural behaviour of probabilistic language models**.

👉 That's why **prompt engineering and RAG exist**.

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

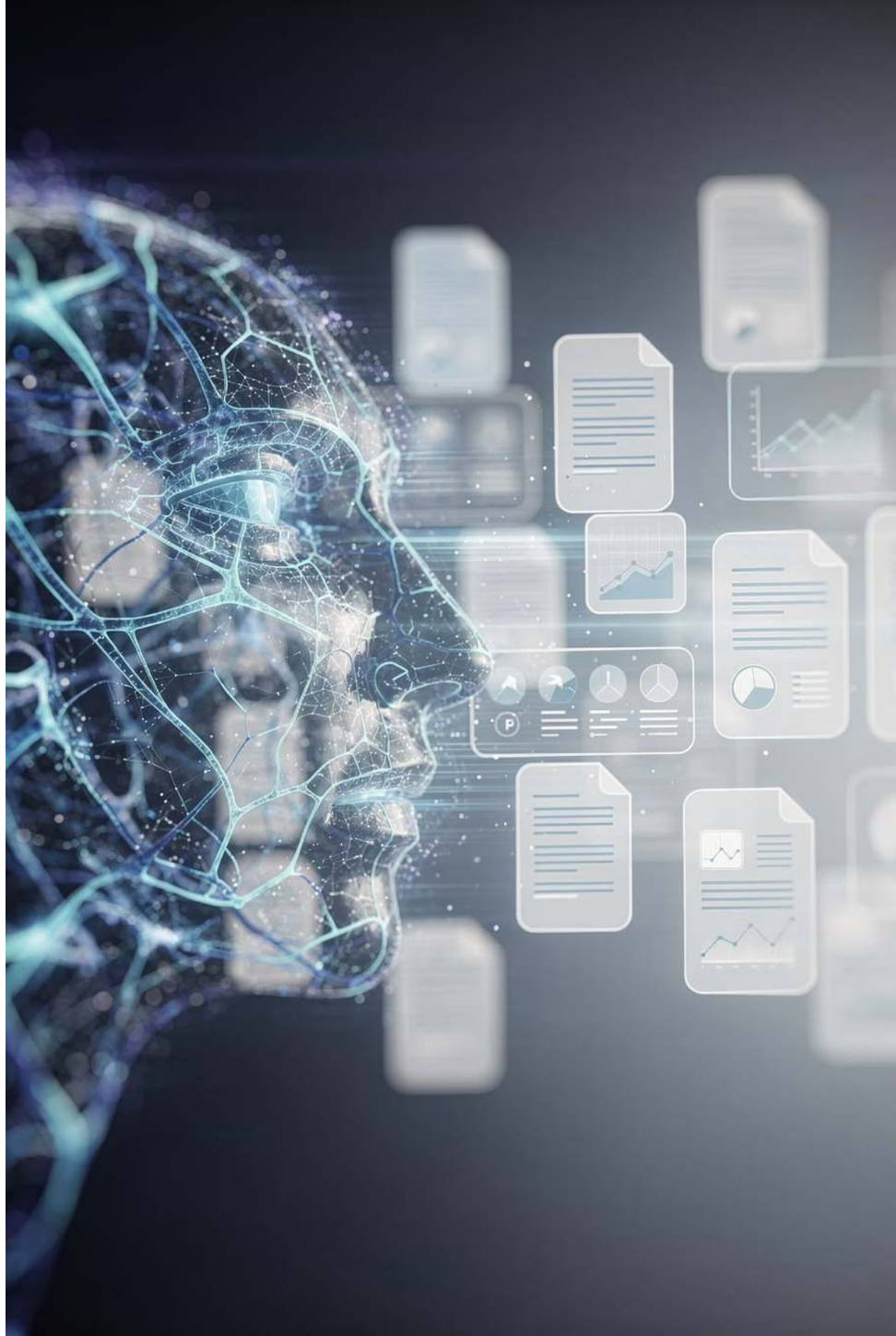
RAG Introduction

Lesson

[Notes] GenAI L9 - RAG Introduction

Retrieval Augmented Generation (RAG)

Making LLMs Accurate Using External Knowledge



Why RAG Exists

Problem with LLMs

- LLMs generate answers from training data
- They do **not verify facts**
- They can hallucinate
- They cannot access private or latest data

Need

👉 A way to ground LLM responses in real data

What is RAG?

Definition

RAG is a technique where an LLM retrieves relevant information from an external knowledge source and uses it to generate grounded, accurate responses.

In Simple Terms

LLM + Search + Your Data = Better Answers

RAG vs Normal LLM

Without RAG

- Answers based only on training data
- Higher hallucination risk
- No access to private documents

With RAG

- Answers grounded in retrieved data
- Lower hallucination
- Can use PDFs, docs, databases



High-Level RAG Architecture

RAG Flow

1

User asks a question

2

Relevant documents are retrieved

3

Retrieved content is added to the prompt

4

LLM generates answer using this context

Key Idea

👉 LLM answers using provided knowledge, not memory

Core Components of RAG

Main Components

- Data source (PDFs, docs, web pages)
- Chunking mechanism
- Embedding model
- Vector database
- Retriever
- LLM (generator)

Each component has a clear role.

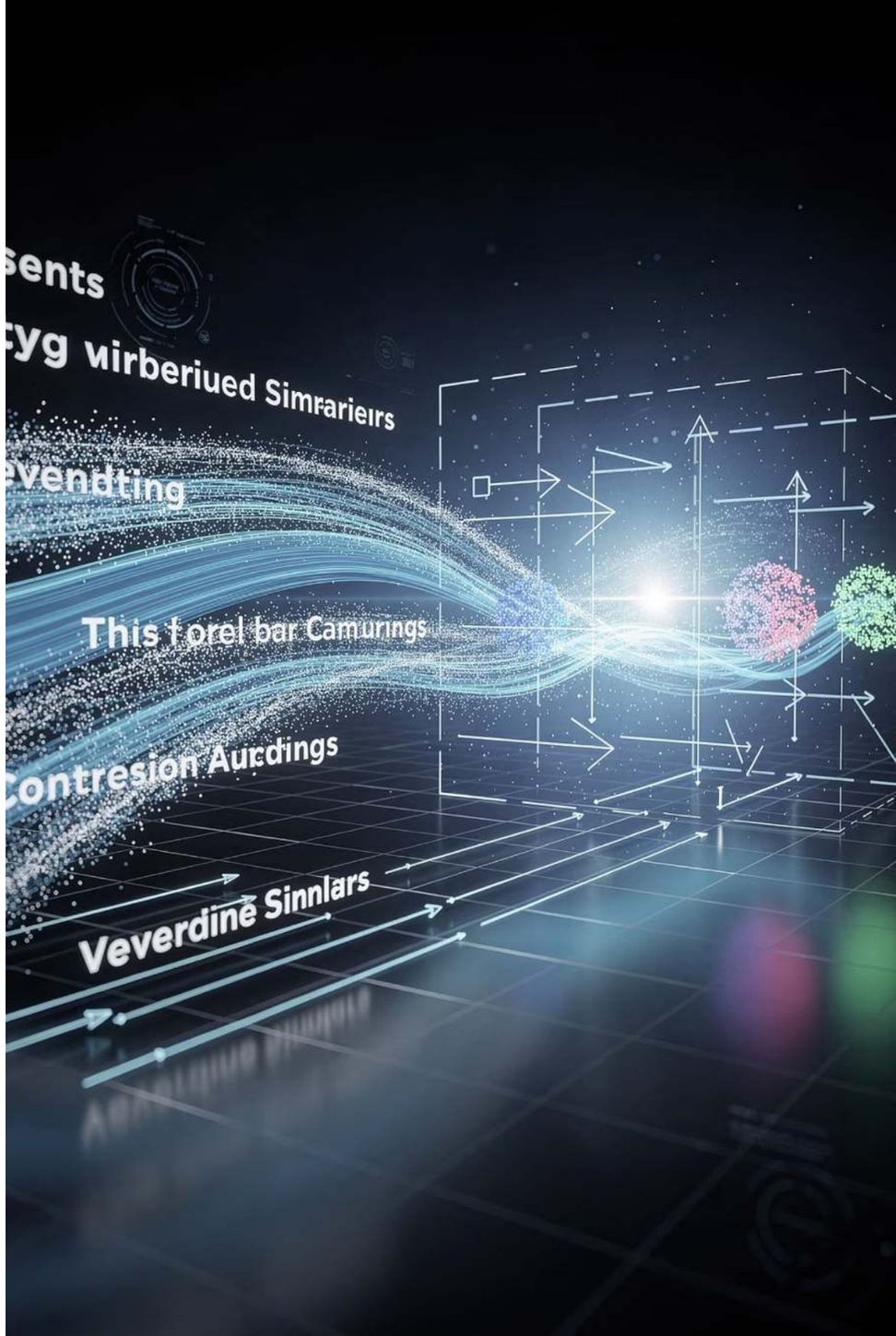
Why Embeddings Are Critical in RAG

Role of Embeddings

- Convert text into vectors
- Enable semantic search
- Retrieve meaning-based results, not keyword matches

Example

"salary details" ≈ "compensation structure"



How RAG Reduces Hallucination

Key Reason

- LLM is forced to answer using retrieved context
- No need to invent missing information

Important Note

RAG **reduces** hallucination

❌ Does not completely eliminate it



RAG vs Prompting vs Fine-Tuning

Prompting

- Controls behaviour
- No new knowledge added

RAG

- Adds external knowledge
- Best for private & dynamic data

Fine-Tuning

- Changes model behaviour
- Expensive
- Not ideal for frequently changing data

When Should You Use RAG?

Use RAG When

- **Data is private (company docs, PDFs)**
- **Data changes frequently**
- **Accuracy is critical**
- **Hallucination is unacceptable**

Examples

- Internal company chatbot
- Policy / compliance assistant
- PDF Q&A system
- Knowledge-base assistant



Key Takeaway

RAG turns LLMs from "smart guessers" into "knowledge-grounded assistants."

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

Langchain

Lesson

[Notes] GenAI L13 - Langchain Introduction

What is LangChain?

- Open-source Python framework

What is LangChain?

- Open-source Python framework
- Build applications powered by LLMs

What is LangChain?

- Open-source Python framework
- Build applications powered by LLMs
- Unified interface for multiple LLM providers

What is LangChain?

- Open-source Python framework
- Build applications powered by LLMs
- Unified interface for multiple LLM providers
- Not a model – an orchestration framework

What Problem Does LangChain Solve?

- Different SDKs for each LLM provider

What Problem Does LangChain Solve?

- Different SDKs for each LLM provider
- Different APIs and invocation styles

What Problem Does LangChain Solve?

- Different SDKs for each LLM provider
- Different APIs and invocation styles
- Tightly coupled application code

What Problem Does LangChain Solve?

- Different SDKs for each LLM provider
- Different APIs and invocation styles
- Tightly coupled application code
- Difficult to switch models later

Why Not Call LLM APIs Directly?

- Multiple SDKs to manage

Why Not Call LLM APIs Directly?

- Multiple SDKs to manage
- Code rewrites when switching models

Why Not Call LLM APIs Directly?

- Multiple SDKs to manage
- Code rewrites when switching models
- Harder to scale real-world systems

Why LangChain?

- Single consistent way to use LLMs

Why LangChain?

- Single consistent way to use LLMs
- Easy switching between providers

Why LangChain?

- Single consistent way to use LLMs
- Easy switching between providers
- Cleaner and maintainable architecture

Why LangChain?

- Single consistent way to use LLMs
- Easy switching between providers
- Cleaner and maintainable architecture
- Saves development time

Big Picture – Mental Model

- LangChain sits between app and LLMs

Big Picture – Mental Model

- LangChain sits between app and LLMs
- Abstracts provider-specific logic

Big Picture – Mental Model

- LangChain sits between app and LLMs
- Abstracts provider-specific logic
- Enables flexibility and vendor independence

Big Picture – Mental Model

- LangChain sits between app and LLMs
- Abstracts provider-specific logic
- Enables flexibility and vendor independence
- Foundation for production LLM apps

Module

Artificial Intelligence

Topic

AI & ML by Arun Sir

Subtopic

RAG using Langchain

Lesson

[Notes] GenAI L20 - Understanging RAG Architecture

Retrieval Augmented Generation (RAG)

Understanding RAG

LLM Challenges & RAG Solutions

Traditional LLMs

Large Language Models (LLMs) face key limitations when relying only on pre-trained data without external knowledge access during inference.

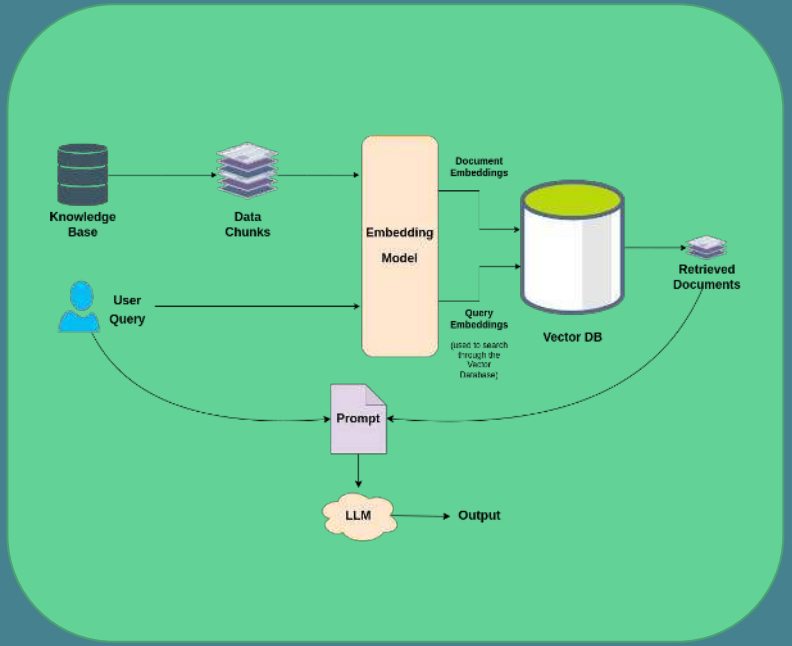
- May produce incorrect or fabricated information when answers are missing
- Often provide outdated, generic, or low-specificity responses
- Generate outputs based on non-verifiable or non-authoritative sources

RAG Solutions

Retrieval Augmented Generation (RAG) overcomes these challenges by connecting LLMs to trusted external knowledge sources in real-time.

- Ensures access to the most current and accurate information
- Supports source attribution and fact verification
- Eliminates costly retraining while maintaining scalability and reliability

About RAG



Retrieval Augmented Generation

Core Concept

RAG enhances large language model outputs by retrieving and grounding them in authoritative external knowledge sources, improving accuracy, reliability, and factual consistency.

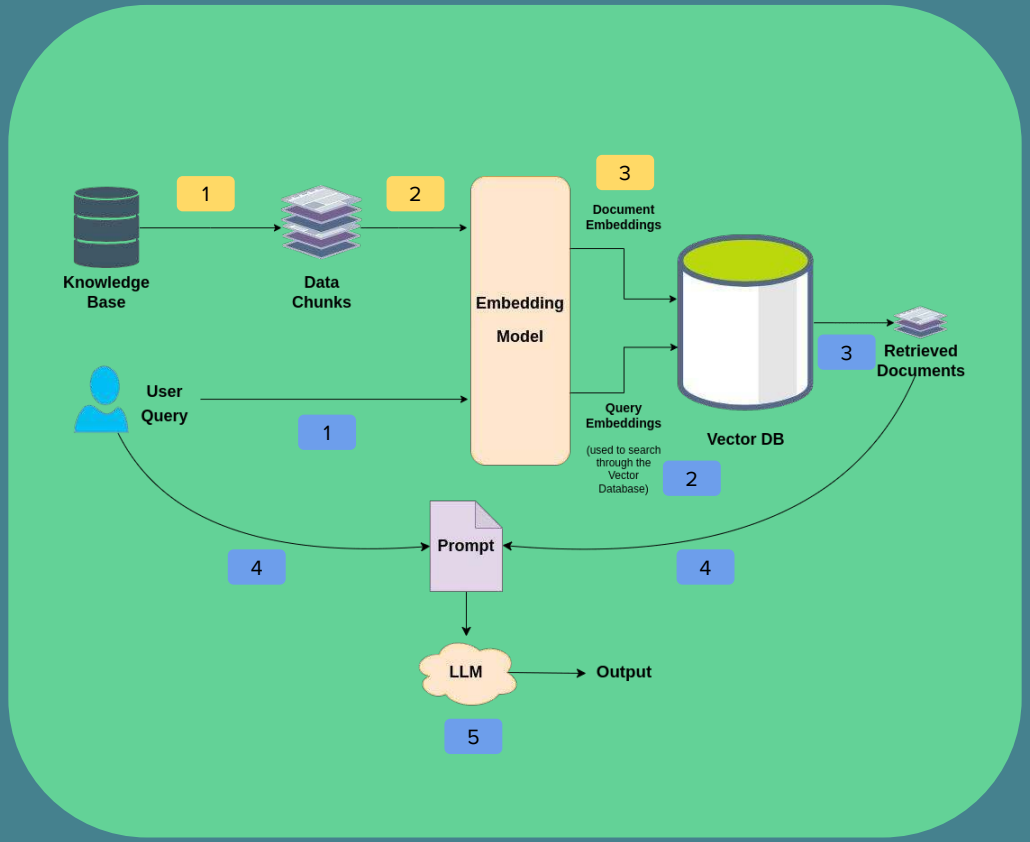
Key Benefits

RAG brings in up-to-date, domain-specific knowledge for relevant answers.It reduces hallucinations and improves reliability.
It also ensures source traceability across domains.

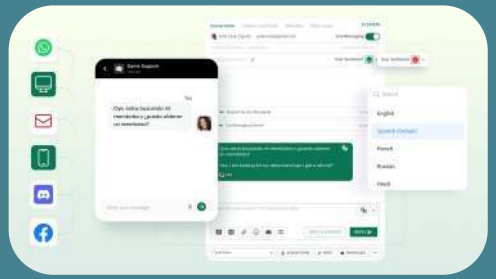
RAG Architecture

Document Ingestion

Retrieval & Generation



RAG Applications



Customer Support

Automated chatbots provide accurate company-specific answers from knowledge bases and documentation

Knowledge Management

Internal Q&A systems enable employees to access HR policies compliance documents efficiently





Search Enhancement

LLM-augmented search engines provide contextual answers alongside traditional search results

Vector DataBases

Database	Type	Hybrid	Perf	Best For
Pinecone	Cloud	Limited	🔥🔥🔥	Scale, no ops
Weaviate	OSS+Cloud	Yes	🔥🔥🔥	Hybrid search
Milvus	OSS+Cloud	Partial	🔥🔥🔥🔥	Billion data
Qdrant	OSS+Cloud	Partial	🔥🔥🔥	Fast + filters
pgvector	OSS (SQL)	SQL	🔥🔥	In-DB vectors
Elastic/Open	OSS+Cloud	Yes	🔥🔥🔥	Enterprise use
Redis	OSS+Cloud	Basic	🔥🔥	In-memory
Vespa	OSS+Cloud	Yes	🔥🔥🔥	Complex rank
ChromaDB	OSS	Basic	🔥	Prototype

RAG System in Python using Langchain